

AI-Powered Humanoid Robot Fleet Management System

Business Rules and Justifications

Rule 1: Minimum Battery to Execute Task.

Purpose: Prevent robots from executing tasks when battery is critically low.

Description: A robot shall not start a task execution if its battery level is below 20%.

Challenges: checking latest timestamp battery level per robot

Assumptions: Latest EnergyStatus record represent current battery

Planned Approach: Trigger on task execution

Justification:

A robot operating with critically low battery may fail mid-task, resulting in incomplete operations and inconsistent system records. Enforcing this rule at the database level ensures that unsafe task executions cannot be inserted regardless of which application interacts with the system.

Relation to Database Design and Earlier Milestones:

In Milestone 2, battery information was normalized into the EnergyStatus table and task execution into TaskExecutions, linked through TaskAssignments. While normalization separated battery status and task execution into distinct entities to remove redundancy, it did not enforce operational safety constraints across those entities. This rule adds cross-table validation logic at the database level.

Rule 2: Charging Station Capacity Enforcement

Purpose: Prevent from overloading the charging stations.

Description: The number of active Charging Sessions (robots) in charging stations shall not exceed its capacity.

Challenges: Count the number of active sessions and the capacity of the stations.

Assumptions: Active session records represent the latest number of robots at charging stations.

Planned Approach: Trigger on ChargingStationsSessions.

Justification:

Charging stations have physical limitations. Allowing more robots than capacity creates unrealistic system states. This rule ensures the database reflects real-world resource constraints.

Relation to Database Design and Earlier Milestones:

Charging infrastructure was normalized into ChargingStations, ChargingSessions, and ChargingStationSessions. While relationships were properly defined, capacity constraints were not inherently enforced. This rule introduces real-world resource validation on top of the normalized design.

Rule 3: One active task per Robot

Purpose: Prevents overloading robots.

Description: A robot can only have one active task at a time and must finish the task before getting another active task.

Challenges: Can cause race conditions with other robots trying to take roles that are dependent on other roles.

Assumptions: a robot can not be multitasking.

Planned Approach: Triggers on Task Execution.

Justification:

A robot cannot realistically perform multiple tasks simultaneously. Without enforcement, conflicting execution states could be recorded. This rule ensures logical consistency and workload integrity.

Relation to Database Design and Earlier Milestones:

Tasks were normalized into Tasks, TaskAssignments, and TaskExecutions. While referential integrity was ensured, exclusivity of active tasks was not enforced structurally. This rule introduces state-based enforcement.

Rule 4: Safe Task Scheduling and Robot Coordination

Purpose: Ensure that robots execute tasks in a coordinated and conflict-free manner within shared environments.

Description: The system must manage task assignments so that each robot is scheduled with a start time that does not conflict with its own existing tasks. A robot shall not begin a new task if it already has an active task in progress. Task execution must be coordinated to prevent overlapping operations for the same robot.

Challenges: Each task execution has a defined start time and end time. A task is considered active if it has started but not yet completed.

Assumptions: Robots operate in shared environments where conflicts are possible. Each task has a defined time window and location. Robot assignments and schedules are stored in the database.

Planned Approach: Use a stored procedure to assign tasks that checks for scheduling conflicts before inserting a new task execution record. Implement triggers to validate

that new or updated task schedules do not overlap with existing ones for the same robot or location. Use helper functions to detect time and location conflicts between tasks.

Justification:

Scheduling overlaps may cause operational conflicts and inefficient task management. Enforcing scheduling validation ensures temporal consistency and prevents overlapping task windows.

Relation to Database Design and Earlier Milestones:

Milestone 2 separated time-based attributes (StartTime, EndTime) within TaskExecutions. This rule builds upon that design by enforcing temporal constraints that cannot be captured by foreign keys alone.

Rule 5: Maintenance Restricts Operational Activity

Purpose: Prevent unsafe operation of robots undergoing maintenance

Description: The system restricts task execution for robots that have an open or active MaintenanceRecord.

Challenges: The database must determine whether a maintenance period is currently active

Assumptions: MaintenanceRecord includes timestamps that indicate maintenance duration.

Planned Approach: Use a trigger on TaskExecution to validate maintenance status before allowing execution.

Justification:

Robots under maintenance should not perform tasks. Without this enforcement, unsafe operational states could be recorded in the database.

Relation to Database Design and Earlier Milestones:

MaintenanceRecord was normalized separately from TaskExecutions. While the schema allows linking robots to maintenance events, it does not restrict execution automatically. This rule enforces operational safety constraints across normalized entities.

Rule 6: Robot cannot charge and execute a task at the same time

Purpose: Prevents robots from having multiple states at once.

Description: A robot can not be doing a task while charging at the same time.

Challenges: A robot might need to start charging while still working on a task.

Assumptions: A robot must stop a task execution before starting to charge.

Planned Approach: triggers on TaskExecutions and ChargingSessions.

Justification:

Charging and task execution are mutually exclusive physical states. This rule ensures logical state consistency within the database.

Relation to Database Design and Earlier Milestones:

Charging sessions and task execution were modeled independently in Milestone 2. This rule enforces cross-state validation between ChargingSessions and TaskExecutions.

Rule 7: Single Active Role per Robot

Purpose: Ensure operational clarity and prevent conflicting behaviour by restricting each robot to one active role at a time.

Description: Single role for the robot so that robot cannot hold more than one active role simultaneously. When a new role assignment is recorded for a robot, any previously active role must be completed, deactivated, or replaced before the new role becomes effective. This prevents ambiguity in robot responsibilities and maintains consistent task coordination.

Challenges: Role records may be updated or inserted over time. The database must determine which role is currently active and prevent overlapping active role assignments for the same robot.

Assumptions: Robot roles are stored in the RobotRole table. A role can be identified as active based on status or timestamp attributes.

Planned Approach: Use a trigger on Robot insertion or update that checks for existing active role records for the same robot. If another active role exists, the operation is rejected or the previous role is automatically deactivated before allowing the new assignment.

Justification:

Allowing multiple active roles could create ambiguity in behavior and responsibility. Enforcing single-role assignment maintains operational clarity.

Relation to Database Design and Earlier Milestones:

Roles were normalized into RobotRole and linked to Robots. However, normalization alone does not prevent overlapping role assignments. This rule adds behavioral enforcement on top of relational structure.

Rule 8: Automatic Alert on Critical Battery Level

Purpose: Ensure low-energy conditions are recorded immediately.

Description: If a robot's battery level falls below a defined threshold, an alert record is automatically generated.

Challenges: Battery values change frequently and must be monitored consistently.

Assumptions: EnergyStatus records battery levels with timestamps.

Planned Approach: Use a trigger on EnergyStatus insert/update to create an Alert when battery level < threshold.

Justification:

Low battery conditions must trigger immediate system awareness. Automated alert generation ensures critical conditions are not missed.

Relation to Database Design and Earlier Milestones:

EnergyStatus and Alerts were modeled separately. This rule introduces automated event-driven logic between normalized entities.

Rule 9: Task Execution Requires Valid Assignment

Purpose: Ensure that robots only execute tasks that have been formally assigned and authorized within the system.

Description: The system must restrict task execution to tasks that have an existing and valid assignment record. A robot cannot begin execution of a task unless that task is associated with a corresponding Task assignment entry linking the robot and the task. This ensures controlled task management within the fleet.

Challenges: Task execution records may be inserted directly without verifying whether a valid assignment exists. The database must validate the relationship between Robot, Task, and TaskAssignment before allowing execution to proceed.

Assumptions: All valid task assignments are recorded in the TaskAssignment table prior to execution. A task execution is considered valid only if a matching assignment exists for the same robot and task.

Planned Approach: Implement a insert trigger on the TaskExecution table that checks for the existence of a corresponding TaskAssignment record for the same robot and task. If no valid assignment is found, the operation is rejected. A helper function may be used to verify assignment validity.

Justification:

We only want a robot to perform a task that it is assigned. The purpose of this is to prevent robot executions from colliding. Spatial reasoning is not needed to make sure the robots and database are coordinated.

Relation to Database Design and Earlier Milestones:

This is directly related to concepts covered in class such as database integrity and user interactions. Once again using a trigger keeps the database safe instead of risking front-end bugs creating a backlog.

Rule 10: Robot performance record and diagnostic report

Purpose: Ensure that declining robot performance is automatically detected and formally documented for further analysis

Description: The system must monitor performance metrics recorded for each robot. When a newly inserted performance metric falls below a defined acceptable threshold for a given metric type, a corresponding diagnostic report is automatically generated for that robot. This ensures that abnormal or degraded performance conditions are not overlooked and are formally tracked within the system.

Challenges: Performance metrics may vary by metric type and threshold values may differ depending on operational context. The database must evaluate the inserted metric value against the appropriate threshold and determine whether a diagnostic report already exists for the same condition to avoid duplicate reports.

Assumptions: Performance metrics are recorded in the PerformanceMetric table with a metric type, value, and timestamp. Threshold values are predefined and consistently applied for each metric type. Diagnostic reports are stored in the DiagnosticReport table and associated with a specific robot.

Planned Approach: Implement a insert trigger on the PerformanceMetric table that compares the new metric value against the defined threshold. If the value falls below the acceptable limit, the trigger inserts a new record into the DiagnosticReport table for the corresponding robot. A helper function may be used to evaluate threshold logic and prevent duplicate reports within a defined time window.

Justification

Performance metrics are continuously recorded for each robot in the PerformanceMetric table. If a robot's metric falls below a certain threshold, a diagnostic report must be generated to flag potential issues. Without enforcing this rule at the database level, low-performance conditions could go unnoticed or inconsistently handled across applications.

The use of a trigger ensures that every inserted metric is automatically evaluated against predefined thresholds stored in MetricThreshold. Additionally, the rule prevents duplicate diagnostic reports within a short time window, reducing redundancy and alert fatigue while maintaining accurate issue tracking.

Relation to Database Design and Earlier Milestones

This rule directly involves:

PerformanceMetric

MetricThreshold

DiagnosticReport

Robots

In Milestone 2, performance metrics and diagnostic reports were normalized into separate entities linked to robots. Threshold values were also abstracted into their own table (MetricThreshold) to allow flexibility and scalability.

Because threshold evaluation depends on comparing values across tables, this rule extends the normalized schema by adding procedural logic (function + trigger). It demonstrates how Phase 2 builds on relational design to enforce dynamic, data-driven business rule.

Rule 11 :Support Request on Critical Alert

Purpose: Ensure that critical robot issues are formally tracked and escalated for resolution.

Description: When an alert with severity marked as “Critical” is recorded for a robot, the system automatically creates a corresponding SupportRequest linked to that robot. This ensures that serious operational issues are not ignored and are formally assigned for review and resolution.

Challenges: The system must detect the severity level at the time of alert creation and prevent duplicate support requests for the same unresolved issue.

Assumptions: Alert records include a severity attribute. A SupportRequest represents a formal record of required maintenance. Technicians are registered in the system and may later be assigned to the support request.

Planned Approach: Implement an insert trigger on the Alert table. When a new alert is inserted with high severity, trigger automatically inserts a corresponding record into the SupportRequest table.

Justification

Alerts generated by robots may vary in severity, but critical alerts require immediate attention. This rule ensures that when a critical alert is recorded, a corresponding support request is automatically created.

Without database enforcement, critical issues might not always trigger support workflows, especially if different applications interact with the database. The trigger guarantees that every critical alert results in an actionable support request while also preventing duplicate open requests for the same robot, ensuring efficient issue management.

Relation to Database Design and Earlier Milestones

This rule directly involves:

Alert
SupportRequest
Robots

During Milestone 2, alerts and support requests were modeled as separate but related entities. Alerts capture system events, while support requests track human intervention processes.

Since there is no inherent constraint in the schema preventing duplicate or missing support requests, this trigger adds behavioral enforcement on top of the relational model. It ensures consistency between operational data (alerts) and service workflows (support requests), illustrating how business logic integrates with normalized tables.

Rule 12 :Emotional record mapping only during interaction session

Purpose: Ensure emotional data is recorded only within valid robot-user interaction periods.

Description: An EmotionalRecord can only be created if it is associated with an existing and active InteractionSession. The timestamp of the emotional record must fall within the start_time and end_time of the corresponding interaction session. This ensures emotional data reflects actual interaction activity rather than standalone or invalid entries.

Challenges: The database must verify temporal consistency by comparing the emotional record timestamp against the interaction session's time window. It must also ensure that the referenced interaction session exists.

Assumptions: InteractionSession stores start_time and end_time. EmotionalRecord includes a timestamp and references an interaction_id. An interaction is considered valid only within its defined time range.

Planned Approach: Use a trigger on the EmotionalRecord table to validate temporal and referential consistency before allowing a new record to be created. The trigger checks that the referenced InteractionSession exists and retrieves its start_time and end_time values. It then compares the EmotionalRecord timestamp against this time window. If the emotional record timestamp falls outside the interaction session's defined time range, the operation is rejected. This ensures emotional data is recorded only during valid interaction periods and maintains temporal integrity within the database.

Justification

Emotional records must only be captured during valid interaction sessions. If emotional data is recorded outside the session timeframe, it could lead to misleading analytics and incorrect interpretations of robot-human interactions.

Relation to Database Design and Earlier Milestones

This rule directly involves:

- EmotionalRecord
- InteractionSession

In Milestone 2, interaction-related data was normalized by separating interaction sessions and emotional records into distinct tables. The EmotionalRecord table references InteractionSession through a foreign key, ensuring that each emotional record is linked to a valid session.

However, while the foreign key constraint guarantees referential integrity, it does not enforce temporal consistency between the two entities. Specifically, it cannot ensure that an emotional record's timestamp occurs within the corresponding session's start and end times.

This rule builds on the normalized design by introducing a BEFORE INSERT trigger that validates time-based constraints across related tables. It ensures that emotional data is not only correctly linked to a session but also recorded within the correct time interval.

Rule 13 : Single Active AI Request per Robot

Purpose: Prevent Overlapping AI processing requests for the same robot and maintain clear decision flow when required.

Description: At any given time, a robot can have only one active AIRequest record. A new AI Request for a robot cannot be created if another request for the same robot is still pending or incomplete.

Challenges: To make the pipeline to get current active AIRequest and associated AIModel.

Assumptions: AI Request includes information to determine whether a request is associated with one model.

Planned Approach: Use an insert trigger on the AIRequest table to check whether an active request already exists for the same robot.

Justification

This rule ensures that AI processing remains controlled and sequential for each robot. Without enforcement at the database level, multiple AIRequest records could be inserted for the same robot simultaneously, resulting in conflicting decisions, duplicated task recommendations, or inconsistent system behavior.

By implementing this logic as a database trigger, the system guarantees that AI request concurrency is managed consistently, regardless of which application or interface submits the request. This preserves data integrity and ensures clear AI decision pipelines.

Relation to Database Design and Earlier Milestones

This rule directly involves:

- AIRequest
- AIResponse

- Robots

In Milestone 2, AIRequest and AIResponse were modeled as separate but related entities linked through foreign keys. While normalization ensured structural integrity, it did not prevent multiple active AIRequests for the same robot.

This rule extends the normalized relational design by adding behavioral enforcement at the database level. It demonstrates how Phase 2 transforms static schema design into dynamic logic enforcement.

Rule 14 :Technician specialization and robot Model

Purpose: Ensure that maintenance tasks are performed only by technicians who are qualified to service a specific robot model.

Description: When a MaintenanceRecord is created, the associated technician has to service the robot . The database prevents assigning wrong technicians.

Challenges: The database must validate that the technician has specialization for the robot model. This requires checking relationships between Technician, Robot, and RobotModels tables.

Assumptions: Technician specializations are stored in an associative table.

Technicians qualifications are maintained and up to date in the database.

Planned Approach:Insert trigger on the MaintenanceRecord table. The trigger retrieves the robot's model and verifies that the technician is associated with that model in the specialization table.

Justification

Maintenance operations directly affect robot safety and operational reliability. Allowing unauthorized technicians to perform maintenance could result in improper servicing, inaccurate logs, or system instability.

Because this validation depends on relationships across multiple tables, it cannot be enforced using simple foreign key constraints alone. Implementing the logic in a trigger ensures that maintenance integrity is enforced consistently at the database layer.

Relation to Database Design and Earlier Milestones

This rule directly involves:

- MaintenanceRecord
- Robots
- Technicians
- DeploymentLocation

During Milestone 2, robots and technicians were normalized into separate entities linked through deployment locations. This relational structure makes it possible to validate technician authorization through shared foreign key relationships.

This rule demonstrates how the normalized schema supports higher-level business constraints by enabling cross-table validation logic in Phase 2.

Rule 15 : Robot Model Deployment compatibility

Purpose: Ensure that different deployment locations operate only with compatible robot models and approved software.

Description: When a robot is assigned to a deployment location, the database validates that the robot model is supported in that specific environment type and that the installed software is approved for that model within that environment. If the combination of location type, robot model, and software is not right, the assignment of the robot to that location is rejected.

Challenges: The validation requires checking across multiple related tables which may increase the complexity.

Assumptions: Certain deployment environments require specific robot models and approved software configurations. For example, hospitals, warehouses, and homes may have different requirements. Each robot model is associated with one or more compatible software and deployment locations define operational constraints based on environment type.

Planned Approach: Implement a stored procedure for robot registration or deployment that validates compatibility between the robot model, software versions and deployment location type before inserting.

Justification

Different operational environments have distinct safety and functional requirements. For example, hospital environments may require specialized robot models that meet medical compliance standards.

Without database-level validation, incompatible deployments could be recorded, leading to inconsistent operational data and safety risks. By enforcing compatibility rules within the database, the system ensures structural and operational consistency.

Relation to Database Design and Earlier Milestones

This rule directly involves:

- Robots
- RobotModels
- DeploymentLocations

In Milestone 2, deployment locations were modeled as a superclass with an EnvironmentType attribute, and RobotModels were separated into their own entity. Robots reference both through foreign keys.

Because compatibility constraints depend on attributes across normalized tables, this rule builds directly upon the relational design decisions made earlier. It demonstrates how Phase 2 extends normalized schema design into active rule enforcement.